Installing the Linux STU-STK on Ubuntu 16.04 LTS

This is a quick guide to setting up the Linux STU-SDK on Ubuntu 16.04 LTS. The steps below were performed on a fresh 64 bit installation of 16.04, and were tested with SDK version v2.10.1 using an STU–530.

Setting up the SDK:

Install JDK:

To run the DemoButtons sample you will first need to install the Java JDK. Run the following command in terminal, entering your password when prompted:

```
sudo apt-get -y install default-jdk
```

Install libusb development library

Communication to the STU devices in Linux is performed via the libusb framework. To install the libusb framework, enter the following command in terminal, entering your password when prompted:

```
sudo apt-get -y install libusb-1.0.0-dev
```

Install OpenSSL development library

Encyption routines in Linux are implemented using OpenSSL. To install the OpenSSL development libraries, enter the following command in terminal, entering your password when prompted:

```
sudo apt-get -y install libssl-dev
```

Setup udev rules

In order to allow connection to the STU device from user space, the udev rules must be configured correctly. Enter the following command in terminal to configure the rules, entering your password when prompted:

```
sudo su -
cat <<EOF >> /etc/udev/rules.d/60-stu.rules
SUBSYSTEM=="usb",ATTR{idVendor}=="056a",MODE:="0666"
SUBSYSTEM=="usb_device",ATTR{idVendor}=="056a",MODE:="0666"
EOF
udevadm control --reload-rules
logout
```

You will now need to unplug and re-connect your STU device for the updated rules to take effect.

Extract the SDK

Finally, you will need to decompress the SDK with the following command in a terminal (this assumes SDK is on desktop):

```
joss@joss-VirtualBox:~$ cd Desktop/
joss@joss-VirtualBox:~/Desktop$ tar -xvjf Wacom-STU-SDK2.10.1.tar.bz2
```

You are now ready to build and run the demo code.

Building and running DemoButtons in Java

To build the DemoButtons sample, firstly you need to navigate to the DemoButtons directory:

```
joss@joss-VirtualBox:~$ cd Desktop/Wacom-STU-SDK/
joss@joss-VirtualBox:~/Desktop/Wacom-STU-SDK$ cd Java/samples/DemoButtons/
joss@joss-VirtualBox:~/Desktop/Wacom-STU-SDK/Java/samples/DemoButtons$
```

Next, you need to build the java using the javac command. As part of the compilation process, you need to specify the location of the wgssSTU.jar file via the cp parameter to javac (this assumes you are in the DemoButtons directory):

```
 javac -cp ../../jar/Linux-x86_64/wgssSTU.jar:. DemoButtons.java
```

Once this is built, you will need to run the sample using the java command line, specifying the class path for the jar file, and setting the java.library.path parameter to the location of the libwgssSTU.so library. assuming you are running the sample from the DemoButtons directory, you can enter the following command:

```
java -cp ../../jar/Linux-x86_64/wgssSTU.jar:. -Djava.library.path=../../jar/Linux-
x86_64 DemoButtons
```

If you have an STU device plugged in, you should see at window with ‘GetSignature’ displayed. Tapping on this button will start the demo buttons signature capture on the STU device, with the capture simultaneously displayed on the Linux screen.

Building and running the C++ samples

You can build and run the C++ samples in either 32 or 64 bit mode by setting the MACHTYPE parameter to i686 for 32 bti builds, or to x86_64 for 64 bit builds. For example, to build the 64 bit C++ library and samples:

```
joss@joss-VirtualBox:~$ cd Desktop/Wacom-STU-SDK/cpp/build
joss@joss-VirtualBox:~/Desktop/Wacom-STU-SDK/cpp/build$ make MACHTYPE=x86_64 -f
gcc46.mak
```

Once the build has completed, the library and samples are located under Linux/${MACHTYPE}:

```
joss@joss-VirtualBox:~/Desktop/Wacom-STU-SDK/cpp/build$ cd Linux/x86_64/
joss@joss-VirtualBox:~/Desktop/Wacom-STU-SDK/cpp/build/Linux/x86_64$ ls
getUsbDevices  i  query  simpleInterface  simpleTablet  WacomGSS.a
joss@joss-VirtualBox:~/Desktop/Wacom-STU-SDK/cpp/build/Linux/x86_64$
```

You can then run getUsbDevices (lists connected STUs), simpleInterface (capture basic pen data) or simpleTablet (capture pen data over an encrypted connection).